Accesar datos

Como vimos en Tipos de Datos, podemos accesar a los elementos de un diccionario usando paréntesis cuadrados con su key adentro.

Este método va a retornar un error si el key no existe.

Para evitar esto, podemos usar el método `get`, que retornará `None` si el key no existe.

Veamos una comparación:

Acceso directo con índices

```
course_information = {
  'title': 'Introduction to DBs',
  'description': 'Here we review the basics
  'length_in_minutes': 600,
}

print(course_information['description'])
```

Here we review the basics of SQL Databases

Acceso usando `get`

```
course_information = {
  'title': 'Introduction to DBs',
  'description': 'Here we review the basic
  'length_in_minutes': 600,
}

print(course_information.get('description'
```

Here we review the basics of SQL Databases

```python
course_information = {
  'title': 'Introduction to DBs',
  'description': 'Here we review the basics',
  'length_in_minutes': 600,
}

print(course_information['episodes'])
```

Copiar

```
Traceback (most recent call last):
File "c:\Users\alekc\repos\d0ad\Secciones\Iteradores
line 7, in <module>
print(course_information['episodes'])
~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^
KeyError: 'episodes'
```

```python
course_information = {
  'title': 'Introduction to DBs',
  'description': 'Here we review the basic
  'length_in_minutes': 600,
}

print(course_information.get('episodes'))
```

Copiar

```
None
```

Iteración

Iterando el `items`

```python
europe_capitals_by_country = {
  'spain' : 'madrid',
  'france' : 'paris',
  'germany' : 'berlin',
  'norway' : 'oslo',
}

for country, capital in europe_capitals_by_country.items():
  print(f'{country} : {capital}')
```

Copiar



```
france : paris
spain : madrid
normway : oslo
germany : berlin
```

También podemos accesar solo a los `keys` o a los `values` de un diccionario usando el método llamado igual a estos.

Los diccionarios no tienen orden!

Iterando el `keys`

Iterando el `values`

```python
europe_capitals_by_country = {
  'spain' : 'madrid',
  'france' : 'paris',
  'germany' : 'berlin',
  'norway' : 'oslo',
}

for country in europe_capitals_by_country.
  print(country)
```

Copiar

spain
france
germany
norway

```python
europe_capitals_by_country = {
  'spain' : 'madrid',
  'france' : 'paris',
  'germany' : 'berlin',
  'norway' : 'oslo',
}

for capital in europe_capitals_by_country.
  print(capital)
```

Copiar

madrid
paris
berlin
oslo

Agregar datos

Debido a que los datos de los diccionarios están atados a keys, debemos accesar a la key que queremos llenar (aunque no exista) y darle un valor:

```python
user_data = {
  'full_name': 'John Snow',
  'email': 'j.snow@gmail.com',
}

user_data['password'] = 'WinterIsComing2023'
print(user_data)
```

Copiar

{'full_name': 'John Snow', 'email': 'j.snow@gmail.com', 'password': 'WinterIsComing2023'}

Eliminar datos

Al igual que con las listas, podemos usar el método `pop` para eliminar el elemento de un key especifico:

Este método también retorna el elemento eliminado.

Copiar

```
student_information = {
  'first_name': 'Harry',
  'last_name': 'Potter',
  'age': 17,
}

deleted_item = student_information.pop('last_name')
print(student_information)
print(f'Deleted item: {deleted_item}')
```

{'first_name': 'Harry', 'age': 17}
Deleted item: Potter